

AI Not Ready to Replace Junior Devs Says Ruby on Rails Creator

Kaustubh Saini



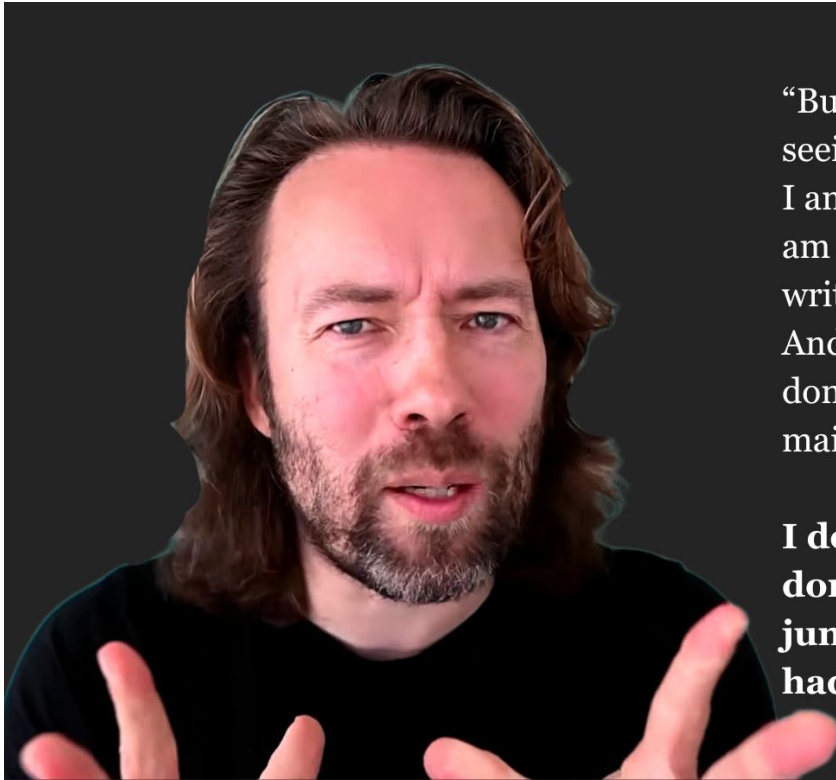
Right now, the popular Silicon Valley narrative is that companies won't need large teams of real programmers anymore. But outside that bubble, many people working in real teams don't see it that way at all.

David Heinemeier Hansson, the creator of Ruby on Rails, believes AI is not as good as most junior programmers for coding, at least as of right now.

Why DHH Thinks AI is Not That Worthy?

David Heinemeier Hansson, also known as DHH, built Ruby on Rails, one of the most influential web frameworks. He still writes code and deals with the messy realities of production. So, when he talks about coding, he is speaking from hands-on experience, not hype.

In the Next Token podcast, David discussed the uncertain impact of AI on the tech landscape.



“But just right now, I am not seeing it. And part of the reason is I am literally looking every day. I am asking AI all the time, can you write this piece of code for me? And it does. And I go like, well, I don't like that. I wouldn't even maintain that.

I don't feel like you have done as good a job as most junior programmers have had to deal with.”

He compared current AI tools to a “flickering” light bulb, something that sometimes gives a useful glimpse but often leaves developers in the dark:

“But just right now, I am not seeing it. And part of the reason is I am literally looking every day. I am asking AI all the time, can you write this piece of code for me? And it does. And I go like, well, I don't like that. I wouldn't even maintain that.

I don't feel like you have done as good as a job as most junior programmers have had to deal with.

And then occasionally I will get the other answer, right? I will ask it for something, and I go like, how did it know? How could it just piece it all together? It's truly remarkable.

I feel like it's a little bit like a flickering light bulb. You are in total darkness, and then it will flicker on, and you go like, I can see everything. And then two seconds later, boom, pitch black. If you can make that light bulb stable and shine all the time, clearly that's amazing for humanity.”

Sounds like he is trying to explain his frustration with AI.

But his skepticism isn't ideological but practical. He is judging AI based on what it can do right now, not what it might do someday.

In David's experience, while the AI usually provides a code that works, the problem is the quality. It's poorly structured or hard to maintain. In real software teams, code is about whether someone else can understand it, fix it, and safely build on it later.

There are times when AI absolutely nails it. He asked a question and got a response so good that it feels almost magical. That's the moment where people get excited and start believing AI is already super capable. But those moments don't last. That's why he uses the flickering light bulb metaphor.

The inconsistency is the real issue.

If AI could produce high-quality results consistently, it would truly change everything. Until then, David sees AI as impressive, but not ready to replace human developers.

In recent years, major industry voices have suggested that AI tools could dramatically reduce the need for human programmers.

One of the first victims of this trend is the freshers. [2026's software engineering job market](#) will have fewer entry-level coding job openings.

But he directly challenges this popular pitch that companies can fire the programmers and still go faster.

Even the AWS CEO, Matt Garman, thinks the same. [Garman said replacing junior developers with AI](#) is “one of the dumbest things” companies can do.

His reasoning is just a little different. He argues junior developers are essential because they grow into senior engineers later, and they already know how to work with AI coding tools today. If companies stop hiring juniors, they create a skills vacuum for the future.

So, AI replacing junior developers sounds convincing on paper, but falls apart in practice.

People think software work is just typing code.

But most of the real effort goes into understanding the problem, designing systems, working with existing code, and dealing with constant changes over time.

That's exactly where AI struggles today. It can spit out code, but it doesn't truly understand the system it's working in or how future changes will affect it.

Junior developers, on the other hand, learn those skills by working with real codebases. They may write imperfect code, but they understand why things were built a certain way.

Companies need to remember that software isn't a one-time task. It evolves. And systems built on code “that somewhat works” tend to collapse over time.

David's point is also that reliability matters more than demos. If a tool is great most of the time but silently wrong some time as well, you start spending time double-checking everything.

That's the hidden cost of code written by the AI coding tools. The debugging and risk management fall back onto humans.

Even a [recent study](#) suggests that while AI assistance can boost output, it can also increase the burden on experienced developers who must review and fix AI-generated code. David's take lines up with that finding.

In his company, Humans wrote 95% of a new product's code

Hansson also said that at his company, 37signals, humans wrote 95% of the code for the new product called Fizzy.

The team tried AI-powered features, but those experiments were not realized in the final work.

This 95% number directly contradicts the idea that serious teams are already letting AI do most of the work. That's why he is amazed by how much economic confidence is riding on AI's assumed ability to replace humans.

While every developer at his company uses it in some form, AI is not writing large features. For a complex task, AI is not helpful for them. It actually slows him down. It can help with quick ideas, but for real production work, humans

are still faster and better right now.

Hansson says he personally continues to write most of his own code. He compared his work to a craftsman enjoying the beauty of fine stitching :

“I still care about how code looks. I still care about the beauty of it. I care about finessing it.

And that’s a luxury that perhaps is akin to what a modern saddle maker enjoys, where the letters are just right, and the stitching is just right, and you are like, okay, but you are no longer part of the main production for transportation.

And I’m like, well, so what, I will continue to make my handwritten code saddles as long as I can for my enjoyment.”

He values both the craft and the precision of manual coding.

Even he shared one moment where the “Vibe Coding” made him pause and question: *Is this actually making me better?*

He caught himself asking AI how to write a basic *if* condition, something he absolutely knows how to do. But he had slowly outsourced that thinking to AI, and now his brain wasn’t reaching for the answer first.

That’s where the doubt kicks in.

AI feels helpful, but it may be weakening core skills. He brought up the fact that some math teachers believed that because of calculators, students didn’t need to memorize times tables anymore. But we now know that’s wrong.

In the same way, if developers stop internalizing basic logic because AI always fills in the gaps, they risk losing intuition.

Bottom Line

Despite his skepticism, Hansson doesn’t think that AI is useless. He points to impressive examples, such as Shopify’s AI tool SiteKick, which helps merchants set up their online stores. In such ways, AI can provide real value when applied to specific tasks.

But when it comes to coding, AI can code, but humans are still needed to understand the system end-to-end, spotting edge cases, debugging, and making final product decisions.

That’s basically what junior developers are trained to grow into. While generative AI can produce snippets or boilerplate code, it still lacks the consistency and reliability of even relatively inexperienced junior devs.

This somewhat lines up with [Nvidia CEO Jensen Huang’s thoughts about coding jobs](#). He argues that writing code isn’t the real purpose of software engineers, but it’s just a task that AI can increasingly handle, while the real job is solving problems. At least, they agree that humans still matter.

So, until AI stops “flickering,” the software world will be run by the people who can verify, integrate, and take responsibility for what ships.